

```

        "amount_inr": random.choice(AMOUNTS_INR),
        "payment_method": random.choice(METHODS),
        "status": "captured",
        "risk_score": random.randint(0, 100)
    })

gateway = pd.concat(
    [gateway, pd.DataFrame(extra_rows)],
    ignore_index=True
)

# Change status for ~2%
status_idx = np.random.choice(
    len(gateway),
    size=int(0.02 * n),
    replace=False
)

gateway.loc[status_idx, "status"] = "failed"

gateway.to_csv("gateway_export.csv", index=False)

print("Data generation completed successfully.")
print(f"Merchants: {len(merchants)}")
print(f"Users: {len(users)}")
print(f"Ledger rows: {len(ledger)}")
print(f"Gateway rows: {len(gateway)}")

```

Overwriting /content/paytm-fintech-capstone/payments_fraud_analytics/generate_data.py

```
%cd /content/paytm-fintech-capstone/payments_fraud_analytics
!python generate_data.py
```

```

/content/paytm-fintech-capstone/payments_fraud_analytics
Data generation completed successfully.
Merchants: 40
Users: 365
Ledger rows: 547
Gateway rows: 530

```

```

import pandas as pd

merchants = pd.read_csv("merchants.csv")
users = pd.read_csv("users.csv")
ledger = pd.read_csv("ledger.csv")
gateway = pd.read_csv("gateway_export.csv")

assert len(merchants) == 40
assert len(users) == 365
assert len(ledger) == 547

burner_count = ledger["transaction_id"].str.startswith("TXN200").sum()
velocity_count = ledger["transaction_id"].str.startswith("TXN300").sum()

assert burner_count == 15
assert velocity_count == 32

print("✅ ALL PART 1 DATA-GENERATION CHECKS PASSED")
print(f"Merchants: {len(merchants)}")
print(f"Users: {len(users)}")
print(f"Ledger: {len(ledger)}")
print(f"Gateway: {len(gateway)}")
print(f"Burner rows: {burner_count}")
print(f"Velocity rows: {velocity_count}")

```

```

✅ ALL PART 1 DATA-GENERATION CHECKS PASSED
Merchants: 40
Users: 365
Ledger: 547
Gateway: 530
Burner rows: 15
Velocity rows: 32

```

```

from google.colab import files

files.download("merchants.csv")
files.download("ledger.csv")

```

```
import os
```

```
print("users.csv exists:", os.path.exists("/content/users.csv"))
print("merchants.csv exists:", os.path.exists("/content/merchants.csv"))
print("ledger.csv exists:", os.path.exists("/content/ledger.csv"))
```

```
users.csv exists: False
merchants.csv exists: False
ledger.csv exists: False
```

```
print(type(users))
print(type(merchants))
print(type(ledger))
```

```
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.frame.DataFrame'>
```

```
import os
```

```
print("users.csv:", os.path.exists("users.csv"))
print("merchants.csv:", os.path.exists("merchants.csv"))
print("ledger.csv:", os.path.exists("ledger.csv"))
print("gateway_export.csv:", os.path.exists("gateway_export.csv"))
```

```
users.csv: True
merchants.csv: True
ledger.csv: True
gateway_export.csv: True
```

```
from google.colab import files
files.download("users.csv")
files.download("merchants.csv")
files.download("ledger.csv")
files.download("gateway_export.csv")
```

```
import pandas as pd
import sqlite3
import os
```

```
# Load the three files required for Part B
merchants = pd.read_csv("merchants.csv")
users = pd.read_csv("users.csv")
ledger = pd.read_csv("ledger.csv")
```

```
# Check dataset sizes
print("Merchants:", len(merchants))
print("Users:", len(users))
print("Transactions:", len(ledger))
```

```
print("\n--- MERCHANTS COLUMNS ---")
print(merchants.columns.tolist())
```

```
print("\n--- USERS COLUMNS ---")
print(users.columns.tolist())
```

```
print("\n--- LEDGER COLUMNS ---")
print(ledger.columns.tolist())
```

```
Merchants: 40
Users: 365
Transactions: 547
```

```
--- MERCHANTS COLUMNS ---
['merchant_id', 'merchant_name', 'category', 'region']
```

```
--- USERS COLUMNS ---
['user_id', 'signup_date']
```

```
--- LEDGER COLUMNS ---
['transaction_id', 'user_id', 'merchant_id', 'transaction_time', 'amount_inr', 'payment_method', 'status', 'risk_score']
```

```
# =====
# PART B – CREATE NORMALIZED SQLITE DATABASE
# =====
```

```
import sqlite3
```

```
# Create / connect to SQLite database
conn = sqlite3.connect("paytm_payments.db")
```

```

# Enable foreign-key enforcement
conn.execute("PRAGMA foreign_keys = ON")

cursor = conn.cursor()

# Remove old tables if they exist
cursor.execute("DROP TABLE IF EXISTS transactions")
cursor.execute("DROP TABLE IF EXISTS users")
cursor.execute("DROP TABLE IF EXISTS merchants")

# -----
# 1. MERCHANTS TABLE
# -----
cursor.execute("""
CREATE TABLE merchants (
    merchant_id INTEGER PRIMARY KEY,
    merchant_name TEXT,
    category TEXT,
    region TEXT
)
""")

# -----
# 2. USERS TABLE
# -----
cursor.execute("""
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY,
    signup_date TEXT
)
""")

# -----
# 3. TRANSACTIONS TABLE
# -----
cursor.execute("""
CREATE TABLE transactions (
    transaction_id TEXT PRIMARY KEY,
    user_id INTEGER NOT NULL,
    merchant_id INTEGER NOT NULL,
    transaction_time TEXT,
    amount_inr REAL,
    payment_method TEXT,
    status TEXT,
    risk_score REAL,

    FOREIGN KEY (user_id)
        REFERENCES users(user_id),

    FOREIGN KEY (merchant_id)
        REFERENCES merchants(merchant_id)
)
""")

# -----
# INSERT MERCHANT DATA
# -----
merchants.to_sql(
    "merchants",
    conn,
    if_exists="append",
    index=False
)

# -----
# INSERT USER DATA
# -----
users.to_sql(
    "users",
    conn,
    if_exists="append",
    index=False
)

# -----
# INSERT TRANSACTION DATA
# -----
ledger.to_sql(
    "transactions",
    conn,
    if_exists="append",

```

```

        index=False
    )

    conn.commit()

    print("✅ SQLite database created successfully!")
    print("Database: paytm_payments.db")

```

✅ SQLite database created successfully!
Database: paytm_payments.db

```

# =====
# VERIFY DATABASE
# =====

print("Merchants in DB:",
      pd.read_sql_query("SELECT COUNT(*) AS count FROM merchants", conn).iloc[0,0])

print("Users in DB:",
      pd.read_sql_query("SELECT COUNT(*) AS count FROM users", conn).iloc[0,0])

print("Transactions in DB:",
      pd.read_sql_query("SELECT COUNT(*) AS count FROM transactions", conn).iloc[0,0])

```

Merchants in DB: 40
Users in DB: 365
Transactions in DB: 547

```

# =====
# QUERY 1 – CHARGEBACK IMPACT
# =====

query1 = """
SELECT
    COUNT(*) AS chargeback_transactions,
    COUNT(DISTINCT user_id) AS unique_users_affected,
    SUM(amount_inr) AS total_chargeback_amount
FROM transactions
WHERE status = 'chargeback';
"""

result1 = pd.read_sql_query(query1, conn)

print("=== CHARGEBACK IMPACT ===")
display(result1)

```

```

=== CHARGEBACK IMPACT ===

```

	chargeback_transactions	unique_users_affected	total_chargeback_amount
0	28	27	54472.0

```

# =====
# QUERY 2 – BURNER ACCOUNTS
# =====

query2 = """
SELECT
    t.transaction_id,
    t.user_id,
    u.signup_date,
    t.transaction_time,
    t.status,
    t.amount_inr,
    CAST(julianday(t.transaction_time) - julianday(u.signup_date) AS INTEGER)
    AS days_since_signup
FROM transactions t
INNER JOIN users u
    ON t.user_id = u.user_id
WHERE t.status = 'chargeback'
    AND julianday(t.transaction_time) >= julianday(u.signup_date)
    AND julianday(t.transaction_time) - julianday(u.signup_date) < 30
ORDER BY days_since_signup ASC;
"""

result2 = pd.read_sql_query(query2, conn)

print("=== BURNER ACCOUNTS ===")
display(result2)
print("Number of burner-account rows:", len(result2))

```

=== BURNER ACCOUNTS ===

	transaction_id	user_id	signup_date	transaction_time	status	amount_inr	days_since_signup	
0	TXN200006	357	2026-01-19 11:00:00	2026-01-23 11:00:00	chargeback	1999.0	4	
1	TXN200008	359	2026-01-18 22:00:00	2026-01-25 22:00:00	chargeback	2999.0	7	
2	TXN200010	361	2026-01-11 07:00:00	2026-01-20 07:00:00	chargeback	4999.0	9	
3	TXN200001	352	2025-12-31 12:00:00	2026-01-11 12:00:00	chargeback	4999.0	11	
4	TXN200002	353	2026-01-10 14:00:00	2026-01-21 14:00:00	chargeback	1999.0	11	
5	TXN200004	355	2026-01-05 12:00:00	2026-01-16 12:00:00	chargeback	4999.0	11	
6	TXN200005	356	2026-01-18 07:00:00	2026-01-29 07:00:00	chargeback	2999.0	11	
7	TXN200000	351	2026-01-15 06:00:00	2026-01-30 06:00:00	chargeback	1999.0	15	
8	TXN200011	362	2026-01-08 02:00:00	2026-01-23 02:00:00	chargeback	4999.0	15	
9	TXN200012	363	2026-01-06 17:00:00	2026-01-23 17:00:00	chargeback	999.0	17	
10	TXN200013	364	2026-01-04 22:00:00	2026-01-22 22:00:00	chargeback	999.0	18	
11	TXN200007	358	2026-01-06 05:00:00	2026-01-28 05:00:00	chargeback	999.0	22	
12	TXN200009	360	2025-12-22 13:00:00	2026-01-13 13:00:00	chargeback	1999.0	22	
13	TXN200014	365	2025-12-27 21:00:00	2026-01-18 21:00:00	chargeback	1999.0	22	
14	TXN200003	354	2025-12-29 19:00:00	2026-01-21 19:00:00	chargeback	4999.0	23	

Number of burner-account rows: 15

Next steps:

[Generate code with result2](#)

[New interactive sheet](#)

```
# =====
# QUERY 3 – VELOCITY ATTACKS
# =====

query3 = """
SELECT
    t1.user_id,
    t1.transaction_time AS cluster_start,
    COUNT(*) AS transaction_count
FROM transactions t1
JOIN transactions t2
    ON t1.user_id = t2.user_id
    AND t2.transaction_time >= t1.transaction_time
    AND t2.transaction_time <= datetime(t1.transaction_time, '+10 minutes')
GROUP BY
    t1.user_id,
    t1.transaction_time
HAVING COUNT(*) >= 3
ORDER BY
    t1.user_id,
    cluster_start;
"""

result3 = pd.read_sql_query(query3, conn)

print("=== VELOCITY ATTACKS ===")
display(result3)

print("Number of qualifying velocity clusters:", len(result3))
```

=== VELOCITY ATTACKS ===

	user_id	cluster_start	transaction_count	
0	59	2026-01-09 21:00:00	4	
1	59	2026-01-09 21:01:00	3	
2	73	2026-01-12 09:00:00	4	
3	73	2026-01-12 09:01:00	3	
4	154	2026-01-02 22:00:00	4	
5	154	2026-01-02 22:01:00	3	
6	200	2026-01-01 22:00:00	4	
7	200	2026-01-01 22:01:00	3	
8	229	2026-01-12 12:00:00	4	
9	229	2026-01-12 12:01:00	3	
10	287	2026-01-14 14:00:00	4	
11	287	2026-01-14 14:01:00	3	
12	314	2026-01-02 18:00:00	4	
13	314	2026-01-02 18:01:00	3	
14	345	2026-01-23 09:00:00	4	
15	345	2026-01-23 09:01:00	3	

Number of qualifying velocity clusters: 16

Next steps:

[Generate code with result3](#)

[New interactive sheet](#)

```
# =====
# QUERY 4 - MERCHANT TRANSACTION SUMMARY
# LEFT JOIN + GROUP BY + HAVING
# =====

query4 = """
SELECT
    m.merchant_id,
    m.merchant_name,
    m.category,
    m.region,
    COUNT(t.transaction_id) AS transaction_count,
    COALESCE(SUM(t.amount_inr), 0) AS total_amount
FROM merchants m
LEFT JOIN transactions t
    ON m.merchant_id = t.merchant_id
GROUP BY
    m.merchant_id,
    m.merchant_name,
    m.category,
    m.region
HAVING COUNT(t.transaction_id) > 0
ORDER BY total_amount DESC
LIMIT 10;
"""

result4 = pd.read_sql_query(query4, conn)

print("=== MERCHANT TRANSACTION SUMMARY ===")
display(result4)
```

=== MERCHANT TRANSACTION SUMMARY ===

	merchant_id	merchant_name	category	region	transaction_count	total_amount	
0	39	Merchant_039	entertainment	North	13	22687.0	
1	13	Merchant_013	travel	East	12	18388.0	
2	7	Merchant_007	food_delivery	East	16	15934.0	
3	25	Merchant_025	travel	South	17	15533.0	
4	27	Merchant_027	ecommerce	North	16	13584.0	
5	10	Merchant_010	grocery	West	14	13086.0	
6	29	Merchant_029	ecommerce	North	19	13081.0	
7	17	Merchant_017	grocery	North	13	13037.0	
8	37	Merchant_037	entertainment	South	19	12931.0	
9	19	Merchant_019	grocery	North	14	12836.0	

Next steps:

[Generate code with result4](#)[New interactive sheet](#)

```
# =====  
# QUERY 5 - HIGH-RISK TRANSACTIONS  
# INNER JOIN + WHERE + DISTINCT  
# =====
```

```
query5 = """  
SELECT DISTINCT  
    t.user_id,  
    u.signup_date,  
    t.transaction_id,  
    t.transaction_time,  
    t.amount_inr,  
    t.status,  
    t.risk_score  
FROM transactions t  
INNER JOIN users u  
    ON t.user_id = u.user_id  
WHERE t.risk_score >= 80  
ORDER BY t.risk_score DESC  
LIMIT 15;  
"""
```

```
result5 = pd.read_sql_query(query5, conn)
```

```
print("=== HIGH-RISK TRANSACTIONS ===")  
display(result5)
```

=== HIGH-RISK TRANSACTIONS ===

	user_id	signup_date	transaction_id	transaction_time	amount_inr	status	risk_score	
0	92	2025-01-05 00:00:00	TXN100003	2026-01-29 13:01:00	49.0	captured	100.0	
1	21	2025-04-29 00:00:00	TXN100156	2026-01-04 20:41:00	2999.0	captured	100.0	
2	19	2024-05-01 00:00:00	TXN100389	2026-01-29 05:59:00	49.0	captured	100.0	
3	330	2025-06-20 00:00:00	TXN100436	2026-01-16 01:22:00	799.0	captured	100.0	
4	154	2024-05-29 00:00:00	TXN100498	2026-01-14 08:26:00	499.0	captured	100.0	
5	355	2026-01-05 12:00:00	TXN200004	2026-01-16 12:00:00	4999.0	chargeback	100.0	
6	358	2026-01-06 05:00:00	TXN200007	2026-01-28 05:00:00	999.0	chargeback	100.0	
7	3	2025-06-18 00:00:00	TXN100029	2026-01-05 20:44:00	1499.0	captured	99.0	
8	330	2025-06-20 00:00:00	TXN100289	2026-01-12 17:21:00	1499.0	captured	99.0	
9	177	2024-10-16 00:00:00	TXN100324	2026-01-15 21:33:00	99.0	captured	99.0	
10	327	2025-07-22 00:00:00	TXN100035	2026-01-25 07:53:00	149.0	captured	98.0	
11	216	2024-05-12 00:00:00	TXN100041	2026-01-02 06:53:00	99.0	captured	98.0	
12	166	2024-09-17 00:00:00	TXN100113	2026-01-03 10:16:00	299.0	captured	98.0	
13	110	2024-08-03 00:00:00	TXN100256	2026-01-12 09:11:00	799.0	captured	98.0	
14	322	2024-09-25 00:00:00	TXN100287	2026-01-08 23:42:00	299.0	captured	98.0	

Next steps:

[Generate code with result5](#)[New interactive sheet](#)

```
# QUERY 6 – USERS WITH MULTIPLE CHARGEBACKS
# GROUP BY + HAVING
# =====

query6 = """
SELECT
    user_id,
    COUNT(*) AS chargeback_count,
    SUM(amount_inr) AS total_chargeback_amount
FROM transactions
WHERE status = 'chargeback'
GROUP BY user_id
HAVING COUNT(*) >= 2
ORDER BY chargeback_count DESC, total_chargeback_amount DESC;
"""

result6 = pd.read_sql_query(query6, conn)

print("=== USERS WITH MULTIPLE CHARGEBACKS ===")
display(result6)
```

```
=== USERS WITH MULTIPLE CHARGEBACKS ===
```

	user_id	chargeback_count	total_chargeback_amount
0	328	2	5048.0

```
# =====
# PART B – FINAL VALIDATION
# =====

print("=====")
print("    PART B FINAL VALIDATION")
print("=====")

# 1. Verify database counts
merchant_count = pd.read_sql_query(
    "SELECT COUNT(*) AS count FROM merchants", conn
).iloc[0, 0]

user_count = pd.read_sql_query(
    "SELECT COUNT(*) AS count FROM users", conn
).iloc[0, 0]

transaction_count = pd.read_sql_query(
    "SELECT COUNT(*) AS count FROM transactions", conn
).iloc[0, 0]

print("\n--- DATABASE COUNTS ---")
print("Merchants:", merchant_count)
print("Users:", user_count)
print("Transactions:", transaction_count)

# 2. Verify burner-account seeded rows
burner_seeded = pd.read_sql_query("""
    SELECT COUNT(*) AS count
    FROM transactions
    WHERE transaction_id LIKE 'TXN200%'
""", conn).iloc[0, 0]

print("\n--- BURNER ACCOUNT VALIDATION ---")
print("Seeded burner rows:", burner_seeded)
print("Required: >= 15")
print("PASS:", burner_seeded >= 15)

# 3. Verify velocity-attack seeded rows
velocity_seeded = pd.read_sql_query("""
    SELECT COUNT(*) AS count
    FROM transactions
    WHERE transaction_id LIKE 'TXN300%'
""", conn).iloc[0, 0]

print("\n--- VELOCITY ATTACK VALIDATION ---")
print("Seeded velocity rows:", velocity_seeded)
print("Expected seeded rows: 32")
print("PASS:", velocity_seeded == 32)

# 4. Count distinct users involved in seeded velocity attacks
velocity_users = pd.read_sql_query("""
```

```

SELECT COUNT(DISTINCT user_id) AS count
FROM transactions
WHERE transaction_id LIKE 'TXN300%'
"""', conn).iloc[0, 0]

print("Distinct seeded velocity users:", velocity_users)
print("Expected seeded clusters/users: 8")
print("PASS:", velocity_users == 8)

# 5. Confirm all six required queries produced output
print("\n--- QUERY OUTPUT VALIDATION ---")

queries = {
    "Query 1 - Chargeback Impact": result1,
    "Query 2 - Burner Accounts": result2,
    "Query 3 - Velocity Attacks": result3,
    "Query 4 - Merchant Summary": result4,
    "Query 5 - High Risk Transactions": result5,
    "Query 6 - Multiple Chargebacks": result6
}

for name, result in queries.items():
    print(f"{name}: {len(result)} rows -> PASS")

# 6. Final status
print("\n=====")
print("          PART B VALIDATION COMPLETE")
print("=====")

```

```

=====
          PART B FINAL VALIDATION
=====

--- DATABASE COUNTS ---
Merchants: 40
Users: 365
Transactions: 547

--- BURNER ACCOUNT VALIDATION ---
Seeded burner rows: 15
Required: >= 15
PASS: True

--- VELOCITY ATTACK VALIDATION ---
Seeded velocity rows: 32
Expected seeded rows: 32
PASS: True

```